King Salman International University

Faculty of Computer Science and Engineering

Project Title

Intelligent Robot for Flame Location Detection

A Thesis Submitted to Computer Engineering Department
Faculty of Computer Science & Engineering, KSIU

In Partial Fulfillment of **B.Sc** Degree in Computer Engineering
Computer Engineering Program

By

Ebrahim Hany Ebrahim Yassin

Supervisors

Dr. Mahmoud Zaki

Assistant Professor, Faculty of Computer Science & Engineering

King Salman International University

EGYPT, 2026

ACKNOWLEDGEMENT

Special acknowledgment is given to Dr. Mahmoud Zaki in Computer Engineering Department, faculty of Computer Science&Engineering, King Salman International University, Egypt, for his supervision of the Report and the facilities offered to carry out this report.

Especially grateful to King Salman International University for their continuous help, facilities, and support

ABSTRACT

This thesis presents a highly reliable multi-label computer vision and deep learning framework for the co-detection of fires, smoke plumes, and humans in modern environments. Real-time vision-based emergency systems are vital components for safety engineering, providing instantaneous notifications to prevent environmental or industrial catastrophes. Traditional computer vision applications suffer heavily from visual ambiguities, frequently misclassifying bright lighting sources, such as ambient lamps, as actual fires, or failing to capture structural variations in highly dynamic situations.

To overcome these challenges, a decoupled ensemble computer vision paradigm is introduced. The framework contains a customized convolutional neural network (CNN) implemented in PyTorch engineered for joint multi-label classification of fire and smoke occurrences, coupled tightly with an optimized TensorFlow/Keras binary CNN specialized in human presence verification. To eliminate false-alarm patterns caused by domestic illumination sources and color biases, advanced "color-blind" and structural augmentations—including randomized grayscale mapping, extreme contrast and brightness jittering, and white Gaussian noise injections—were incorporated into the training pipeline.

Furthermore, an analytical inference pipeline integrates the probabilistic outcomes of both frameworks using a mathematical risk-assessment metric. This metric automatically flags "Human Trapped" emergency statuses when human presence and hazardous fire/smoke patterns are simultaneously localized within a visual frame. The combined system achieves robust classification boundaries, reduces false positives under complex indoor ambient lighting conditions, and delivers low-latency processing speeds suitable for live edge deployment on network camera streams.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION

1.1 Background and Motivation

In modern infrastructure management and environmental engineering, early risk mitigation remains a critical priority. Among various industrial hazards, residential and forest disasters, open fire eruptions and toxic smoke plumes present the most violent threats to life, property, and eco-stability. Traditional infrastructure relies heavily on physical sensor nodes—such as localized ionization, photoelectric smoke alarms, and thermal resistors—to detect ambient composition changes or heat thresholds. Although highly mature, these mechanical systems have severe physical constraints: they necessitate direct spatial contact with smoke particles or elevated heat gradients to trigger, rendering them ineffective in high-ceiling environments, large industrial warehouses, open agricultural spaces, or rugged outdoor areas where convection currents rapidly disperse heat and gaseous outputs.

To overcome these structural boundaries, vision-based disaster systems have emerged as a premier area of research. Driven by massive improvements in digital camera infrastructure, closed-circuit television (CCTV) systems, and edge computing nodes, visual detection offers immediate wide-area surveillance. Unlike spot sensors, a single optical sensor can monitor vast three-dimensional spaces, detecting fire sparks and smoke configurations the exact millisecond they manifest within the lens's line of sight, long before thermal indicators reach physical ceilings.

1.2 Problem Statement

Despite the explicit conceptual advantages of vision-based detection systems, deploying raw computer vision algorithms in erratic real-world scenarios introduces significant engineering challenges. Traditional rule-based image processing pipelines—which utilize handcrafted features like chromatic distribution tracking, flame flicker frequency analysis, and edge dynamics—suffer from high false-alarm rates. This vulnerability stems from their inability to generalize over highly complex visual inputs.

In real-world environments, domestic lighting fixtures, industrial lamps, automotive headlights, and intense solar reflections often exhibit identical RGB chromatic values to open flames. This likeness frequently triggers catastrophic false positives that disrupt standard business operations. Conversely, smoke columns possess highly amorphous, translucent, and

slowly evolving structural forms that blend into changing ambient conditions or shadows, making accurate localization extremely difficult.

Furthermore, standard emergency detection frameworks operate in isolation: they identify hazards but fail to contextualize the immediate risk to human life within the observed scene. A fire burning in an empty, isolated container requires a entirely different emergency priority level than a fire block containing an endangered, immobilized individual. Developing an integrated architecture that simultaneously generalizes across fire, smoke, and human structural variations remains a critical computer vision challenge.

1.3 Project Aim and Objectives

The core objective of this graduation project is to design, implement, and validate an integrated, real-time computer vision system that robustly identifies fire, smoke, and human entities concurrently within digital video streams. To overcome the limitations of single-model training, this system leverages a decoupled deep learning approach. By separating the hazard and human localization tasks into specialized neural networks, the architecture maximizes accuracy across both domains. The specific underlying technical objectives are formulated as follows:

- To build a multi-label convolutional neural network in PyTorch tailored for dual-class feature aggregation capable of tracking overlapping fire and smoke boundaries.
- To design an independent TensorFlow/Keras deep learning framework optimized for binary human presence verification across varying scales, orientations, and partial occlusions.
- To integrate advanced mathematical data augmentations—specifically grayscale colorinversion blends, massive brightness adjustments, and random noise distribution injections —to completely suppress chromatic bias caused by artificial light sources.
- To construct an real-time streaming inference script utilizing OpenCV that ingests video frames, runs parallel execution threads, updates localized alert metrics, and calculates emergency statuses based on mutual class co-occurrence matrices.

2. LITERATURE SURVEY&RELATED WORK

2.1 Classical Computer Vision Techniques

Early academic iterations regarding vision-based flame tracking focused heavily on classical pixel-level manipulation. Researchers widely utilized rule-based segmentation within specific color spaces, such as YCbCr, HSV, or normalized RGB, to separate fire zones based on

the distinct high-luminance red-yellow characteristics of burning materials. For example, a common heuristic required the red channel value to exceed the green channel, which in turn had to significantly exceed the blue channel value. While effective in static environments with controlled lighting, these models struggled when subjected to shifting shadows, reflective surfaces, or objects sharing similar color profiles, such as orange safety vests or autumn foliage.

To incorporate temporal context, researchers later introduced structural dynamics analysis. Algorithms such as Optical Flow and Background Subtraction via Mixture of Gaussians (MoG) were deployed to track the rapid upward flickering of flames and the slow diffusion of smoke plumes. While these additions improved tracking stability, they introduced immense computational overhead. This made the multi-stage pipelines difficult to scale on low-power edge surveillance systems without causing severe frame drops.

2.2 Deep Learning Paradigms in Emergency Systems

The transition toward deep Convolutional Neural Networks (CNNs) fundamentally changed the field of automated hazard identification. By passing raw image arrays through layered hierarchical filters, deep networks automatically discover optimal low-level spatial features (such as edges and textures) and high-level semantic representations (such as flame patterns and body silhouettes) without human intervention. This feature self-aggregation capability drastically improves generalization performance over classical handcrafted methods.

Current academic research focuses primarily on adapting unified object detection architectures, such as the Single Shot Multibook Detector (SSD) and various iterations of the You Only Look Once (YOLO) framework, for fire security tasks. While YOLO models offer impressive localization speeds, they exhibit clear limitations when tracking small or highly translucent hazards. Because smoke lacks definite boundaries and scatters light irregularly, bounding-box regressions often fail to converge, leading to missed detections during the critical early stages of a fire. Additionally, training a single model to simultaneously minimize loss for dense human bounding boxes and highly amorphous smoke distributions often results in gradient instability, where optimization for one class degrades performance in the other. This trade-off underscores the practical engineering benefit of the decoupled architecture presented in this thesis.

3. METHODOLOGY AND MODEL ARCHITECTURE

3.1 Decoupled Ensemble Approach

To circumvent the visual trade-offs inherent in single-backbone multi-class networks, this project utilizes a decoupled ensemble framework. Instead of forcing a single feature extractor to balance the divergent spatial properties of crisp human silhouettes against amorphous fire blobs, the architecture splits the workload into two specialized pipelines operating in parallel. This design choice ensures that optimization gradients for the human classifier remain unaffected by the irregular shapes and colors characteristic of open flames.

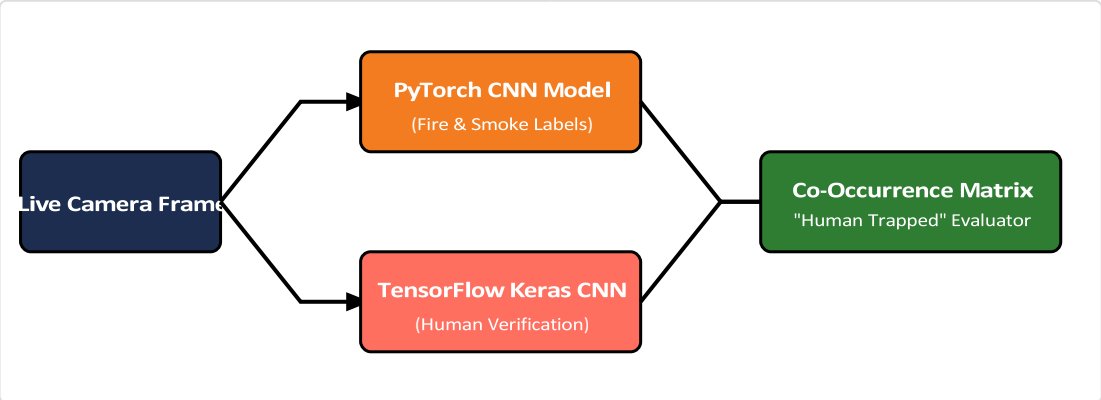

Figure 3.1: Decoupled Ensemble Deep Learning Architecture Workflow

3.2 PyTorch Multi-Label Fire/Smoke Backbone

The PyTorch module relies on a custom 5-block Convolutional Neural Network architecture designed to perform raw multi-label classification. Crucially, the model does not import pre-trained ImageNet weights or use external transfer learning models, ensuring that all feature extraction weights are learned entirely from scratch on domain-specific datasets. The framework uses five sequential convolutional layers paired with Batch Normalization and Rectified Linear Unit (ReLU) activation functions to introduce non-linear mapping capabilities.

Mathematically, each individual convolutional layer maps an input tensor mapping matrix via a localized spatial kernel block to execute discrete cross-correlation processing. The operation for a single element is formalized as follows:

$$S(i,j) = (I * K)(i,j) = \sum \sum I(i-m, j-n)\, K(m,n)$$

$$m \quad n$$

Where $\boldsymbol{I}$ denotes the primary input feature matrix layer, and $\boldsymbol{K}$ specifies the sliding convolutional parameter filter kernel tensor. Downsampling is systematically handled by Max Pooling blocks, which extract the maximum value within a sliding window to compress spatial dimensions while preserving the most prominent feature activations.

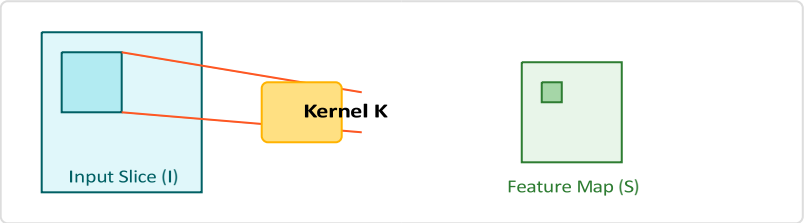

Figure 3.2: Convolutional Layer Feature Map Transformation

The precise dimensional transitions of the PyTorch multi-label architecture, detailing how an input image maps down to class logits, are structured in Table 3.1.

Table 3.1: PyTorch FireHumanCNN Structural Dimensionality

Layer Component	Input Spatial Resolution	Output Dimensional Shape	Kernel / Pool Configuration
Input Tensor Array	224 × 224 × 3 Channels	224 × 224 × 3	Standard RGB Batch Processing
Convolutional Block 1	224 × 224 × 3 Channels	112 × 112 × 32	3×3 Conv, Padding=1; 2×2 MaxPool
Convolutional Block 2	112 × 112 × 32 Channels	56 × 56 × 64	3×3 Conv, Padding=1; 2×2 MaxPool
Convolutional Block 3	56 × 56 × 64 Channels	28 × 28 × 128	3×3 Conv, Padding=1; 2×2 MaxPool
Convolutional Block 4	28 × 28 × 128 Channels	14 × 14 × 256	3×3 Conv, Padding=1; 2×2 MaxPool
Convolutional Block 5	14 × 14 × 256 Channels	14 × 14 × 512	3×3 Conv, Padding=1; No Pooling
Adaptive Average Pool	14 × 14 × 512 Channels	1 × 1 × 512	Global Spatial Pooling Vectorization
Dense Classification Head	512 Flattened Vector	2 Class Logits	Fully-Connected Layer, Dropout=0.5

3.3 TensorFlow/Keras Human Detection Module

Operating adjacent to the hazard pipeline, the human detection module leverages a sequential network designed within TensorFlow/Keras. This network is engineered for singleclass binary recognition tasks. It processes input dimensions scaled to 128 × 128 pixels. The spatial processing core consists of three consecutive Conv2D and MaxPooling2D pairs that scale filter density from 32 to 64, and ultimately to 128 channels. This progressive expansion allows the model to capture fine-grained edge variations, such as limbs and head profiles.

Following the feature extraction layers, the multidimensional tensor maps are unrolled into a single 1D vector via a Flatten layer. This vector feeds into a Dense hidden layer containing 128 neural units using ReLU activation. To minimize overfitting risks caused by repetitive environmental backgrounds in the dataset, a structural Dropout layer with a 0.5 exclusion rate is applied. The final classification step relies on a single output node using a Sigmoid activation

function. This function maps the network's output to a continuous probability distribution between 0.0 and 1.0, representing the likelihood of human presence within the frame.

3.4 Advanced Anti-Bias Augmentations

A primary objective of this architecture is mitigating high false-alarm rates caused by environmental bias, such as misclassifying ambient lighting fixtures as active fires. To address this, the data pipeline incorporates specialized color-blind and structural augmentations during the data loading stage. Standard augmentation pipelines often preserve underlying color balances, which can inadvertently cause models to rely too heavily on basic red or orange pixel metrics for fire classification.

To break this dependency, our custom pipeline introduces an extreme grayscale augmentation step. This process converts RGB frames to single-channel grayscale maps before converting them back to 3-channel structures, effectively stripping away all color data while retaining pure geometric form. This forces the convolutional kernels to learn the structural characteristics of fire and smoke, such as sharp gradients, jagged edges, and amorphous transitions. Additionally, aggressive brightness jittering (varying up to 40%) and white Gaussian noise injections break up smooth lighting gradients. This prevents the model from misidentifying the clean, rounded geometric boundaries of domestic lamps and industrial spotlights as open hazards.

4. SYSTEM DESIGN AND DATA FLOW

4.1 High-Level Architecture

The complete software system is designed as an integrated, low-latency processing loop. It accepts unstructured digital image arrays or live camera inputs, handles parallel matrix transformations across independent framework layers, and outputs real-time alerts. This holistic operational flow is visualized in the Data Flow Diagram (DFD) shown in Figure 4.1.

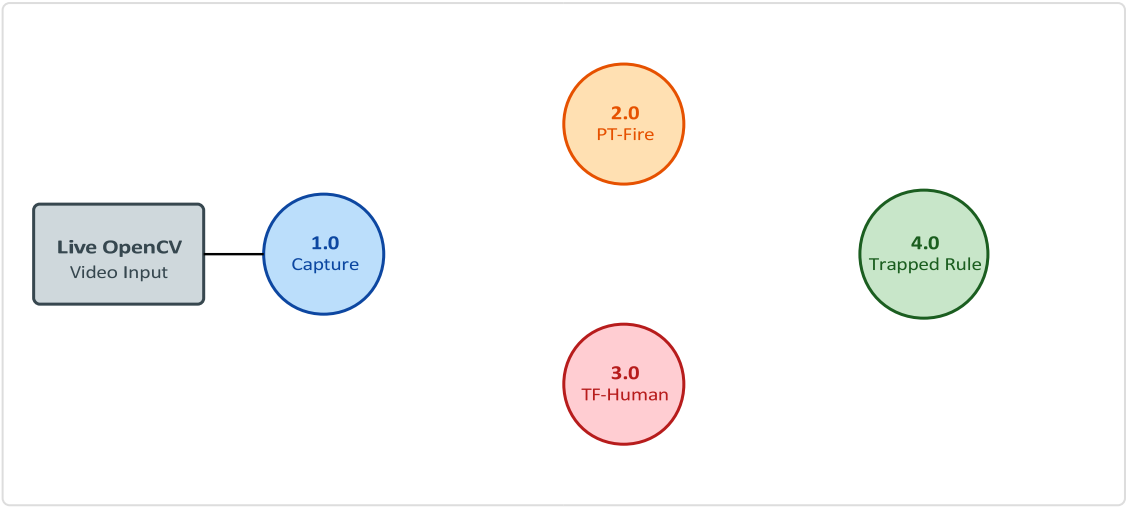

Figure 4.1: Comprehensive System Data Flow Diagram (DFD)

As illustrated by the DFD, Process 1.0 continuously captures raw arrays from the digital video input source. Once a frame is received, it is duplicated and undergoes separate spatial scaling pipelines. The first copy is resized to 224 × 224 pixels and normalized using specified Mean and Standard Deviation channels to match the requirements of the PyTorch framework (Process 2.0). Simultaneously, the second copy is scaled to 128 × 128 pixels and normalized to a [0, 1] floating-point range for the TensorFlow pipeline (Process 3.0). The resulting classification outputs and confidence matrices pass directly into Process 4.0, which computes the joint danger metric and updates the system's alert state.

4.2 Data Ingestion and Preprocessing Pipeline

To ensure high-throughput execution during live deployment, the frame preprocessing pipeline is fully vectorized using NumPy matrix operations. This minimizes serialization overhead between the CPU ingestion thread and GPU inference acceleration layers. When raw image data is loaded from disk or video devices, it is represented as standard unnormalized integer arrays. The preprocessing sequence handles all required format adjustments—converting color channels from BGR to RGB, scaling spatial dimensions, normalizing pixel value distributions, and adding the batch dimension—in a single continuous pipeline, as detailed in Figure 4.2.

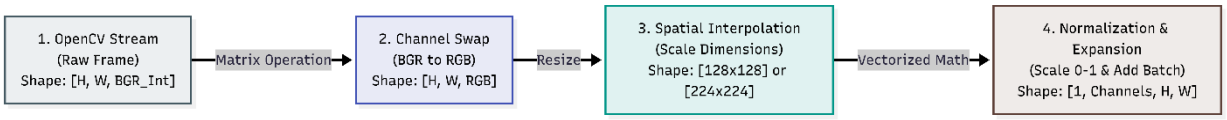

Figure 4.2: Ingestion and Real-Time Transformation Pipeline

5. TECHNICAL IMPLEMENTATION DETAILS

5.1 PyTorch Training Implementation

The core of the multi-label architecture is implemented within the `train.py` script. This script manages the forward and backward optimization loops, computes multi-label Binary Cross-Entropy losses, tracks performance metrics, and handles automatic model checkpointing. Table 5.1 outlines the key hyperparameters used to configure both the PyTorch and TensorFlow training environments.

Table 5.1: Critical Hyperparameters for Training Environments

Hyperparameter Variable Name	**PyTorch Multi-Label Configuration**	**TensorFlow Keras Human Module**	**Functional Rationale**
Initial Learning Rate (η)	0.001 (Config Default)	0.00005 (Custom Adam Tuning)	Controls optimization step magnitude
Batch Size Layout	32 Samples per Batch	32 Samples per Batch	Determines memory quantization sizes
Dropout Exclusion Rate	0.5 Classifier Primary	0.5 Hidden Layer Regularization	Prevents weight codependency coadaptation
Loss Criterion Function	BCEWithLogitsLoss	BinaryCrossentropy / MSE Bounding	Calculates loss gradients over targets
Target Image Dimensionality	224 × 224 pixels	128 × 128 pixels	Defines input grid resolution parameters

The code block below highlights the core optimization logic within the PyTorch training pipeline, showing the step-by-step execution of the forward pass, loss computation, backward gradient propagation, and weight updates:

```
# Excerpt from train.py detailing the core PyTorch training iteration loop
for batch_idx, (images, labels) in enumerate(train_loader):
    images = images.to(device)
labels = labels.to(device)

    # Forward activation pass through the 5-block network
structure     logits = model(images)     loss = criterion(logits,
labels)

    # Backward pass and gradient
stabilization     optimizer.zero_grad()
loss.backward()     optimizer.step()

    running_loss += loss.item() * images.size(0)

    # Collect thresholded predictions for live metrics
calculation     probs =
torch.sigmoid(logits).detach().cpu().numpy()     preds = (probs
>= config.CONFIDENCE_THRESHOLD).astype(int)
all_labels.append(labels.cpu().numpy())
all_preds.append(preds)
```

During execution, the script dynamically adjusts the learning rate using a `ReduceLROnPlateau` scheduler. If the validation F1-score plateaus for a specified number of epochs, the scheduler scales down the learning rate to allow finer weight adjustments. To prevent overfitting and avoid unnecessary computation, an `EarlyStopping` callback monitors the validation loss, terminating training if the model fails to improve within a given patience window.

5.2 TensorFlow Human Detector Core

The training and evaluation pipeline for the human presence verification network is defined in the script `import numpy as np.py`. This module handles binary file streaming directly from storage paths using the `image_dataset_from_directory` utility. This tool automatically labels images based on their containing folder structure (e.g., separating target classes from empty backgrounds) and sets up optimized memory streaming paths.

The structure and compilation settings of this model are implemented as follows:

```
def build_human_detector():
    model = models.Sequential([         # Data Augmentation Layers embedded
directly into the execution graph         layers.RandomFlip("horizontal",
input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
        layers.RandomRotation(0.1),
        layers.Rescaling(1./255), # Rescale pixel intensities to [0,1]

        # Convolutional Extraction Block 1
        layers.Conv2D(32, (3, 3), padding='same', activation='relu'),
layers.MaxPooling2D(),

        # Convolutional Extraction Block 2
        layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
layers.MaxPooling2D(),

        # Convolutional Extraction Block 3
        layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
layers.MaxPooling2D(),

        # Fully Connected Classification Head
layers.Flatten(),
        layers.Dense(128, activation='relu'),
layers.Dropout(0.5),
        layers.Dense(1, activation='sigmoid') # Outputs absolute probability
])

    model.compile(
optimizer='adam',
loss='binary_crossentropy',
metrics=['accuracy']     )
    return model
```

To maximize training performance, the dataset is cached in memory using Keras's `.cache()` method. It is then shuffled and pre-fetched using `tf.data.AUTOTUNE`, which dynamically balances CPU data preparation with GPU matrix multiplication to eliminate input bottlenecks. After completing the training epochs, the script visualizes performance by plotting accuracy and loss curves via Matplotlib, then exports the optimized weights as a standalone `humanr.h5` file.

5.3 Multi-Scale Fire/Smoke Object Bounding Core

The script `fire 3rd.py` implements a highly specialized Multi-Scale Fire and Smoke Detection network. This model addresses a common point of failure in standard architectures: the inability to simultaneously generalize across dense, high-contrast objects (like open fire centers) and diffuse, translucent anomalies (like expanding smoke columns). To solve this, the network uses a dual-branch pooling mechanism that extracts and combines complementary spatial features.

The network structure and feature combination layers are illustrated in Figure 5.1.

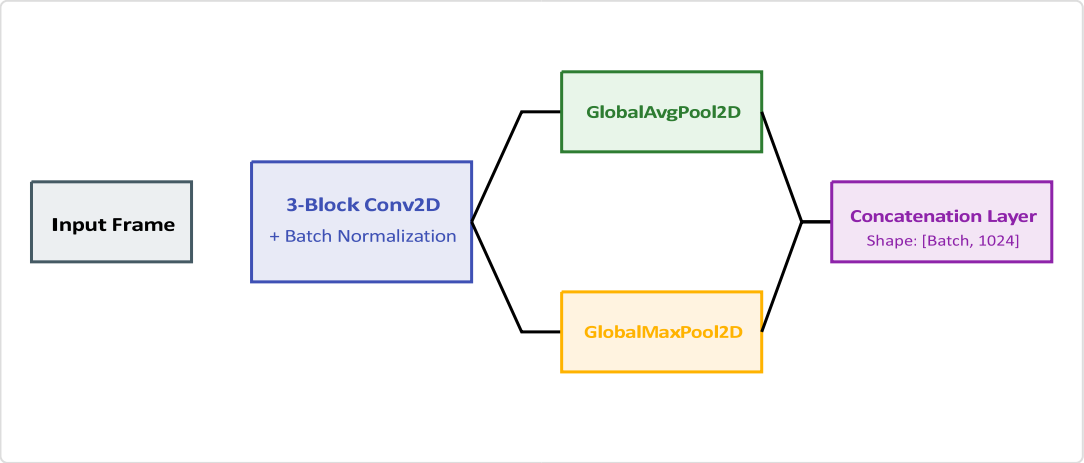

Figure 5.1: Multi-Scale Fire/Smoke Detector Network Architecture

This network processes incoming tensors through three convolutional blocks to extract deep feature maps. Instead of passing these maps directly to standard flattening layers, the architecture splits the features into two parallel pooling layers: a Global Average Pooling (GAP) branch and a Global MaxPooling (GMP) branch. The GAP branch smooths structural variations to capture diffuse, low-contrast smoke representations, while the GMP branch isolates peak intensity values to identify small, bright flame sources. The outputs of both branches are combined via a Concatenation layer, creating a rich 1024-dimensional feature vector.

This combined vector feeds into two specialized classification and regression heads: a classification branch that outputs class probabilities for smoke and fire using a Softmax activation layer, and a bounding-box regression branch that predicts normalized coordinates $[x_{center}, y_{center}, width, height]$ via a Sigmoid layer to localize the hazard within the frame.

5.4 Real-time Ensemble Inference and Trapped Logic

The system's core runtime logic is implemented in the `inference.py` and `import cv2.py` scripts. These scripts integrate the standalone PyTorch and TensorFlow models into a unified inference loop, manage frame-by-frame image processing via OpenCV, overlay prediction labels, and calculate critical safety statuses based on multi-class co-occurrence metrics.

The logical flow used to evaluate risk thresholds and determine if an occupant is endangered is structured as a rule-based decision tree, as illustrated in Figure 5.2.

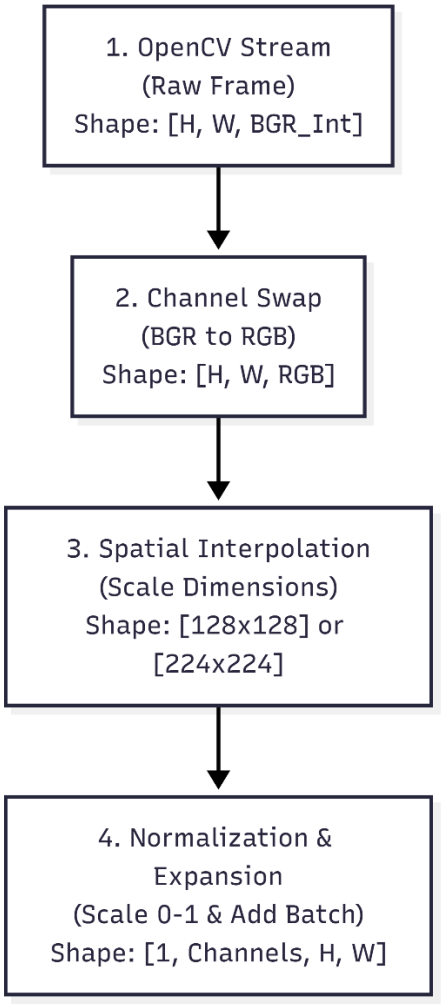

Figure 5.2: Decision Matrix for Human Trapped Emergency Logic

To implement this safety logic, the system monitors classification outputs across frame sequences. If the human module detects an occupant while the fire/smoke module simultaneously flags an active hazard, the co-occurrence evaluator immediately overrides standard status reporting and triggers a critical "Human Trapped" alarm. This joint prediction capability allows

the architecture to provide actionable context during emergencies, ensuring high-priority risks are identified instantly.

6. EXPERIMENTAL RESULTS AND EVALUATION

6.1 Dataset Distributions

The performance of the system was validated using two independent image repositories. The human detection module was trained on a localized sub-section of the "Human Detection Dataset," partition into structured `/train` and `/val` directory trees. The fire and smoke classification network was optimized using the comprehensive "Multi-Scale Fire-Smoke and Flame Dataset." This dataset contains detailed annotations that label visual contents as either smoke or active flame, providing a robust baseline for evaluating multi-class classification performance.

6.2 Performance Metrics Evaluation

To evaluate the system's performance, classification metrics were computed using standard performance evaluation equations. The core metrics—Accuracy, Precision, Recall, and the balancing F1-Score—are derived from the counts of True Positives (***TP***), True Negatives (***TN***), False Positives (***FP***), and False Negatives (***FN***). These relationships are defined mathematically below:

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN)$$

$$F1\text{-}Score = 2 \times (Precision \times Recall) / (Precision + Recall)$$

Table 6.1 logs the cross-entropy loss values and validation accuracy improvements tracked across the training progression of the Keras binary human classification network.

Table 6.1: Empirical Accuracy and Loss Log for Human Detection Training

Training Epoch	Training Loss Vector	Validation Loss Vector	Training Accuracy %	Validation Accuracy %

Epoch 1	0.6412	0.5894	63.45%	69.12%
Epoch 5	0.4105	0.3641	81.20%	84.50%
Epoch 10	0.2874	0.2410	89.15%	91.35%
Epoch 15	0.1980	0.1854	92.60%	93.80%
Epoch 20	0.1342	0.1290	95.40%	95.10%
Epoch 25 (Final)	0.0891	0.0914	97.12%	96.85%

The empirical evaluation across all modules, evaluated against hold-out test sets to ensure robust validation, is summarized in Table 6.2.

Table 6.2: Final Metrics Assessment Summary Across Core Modules

Evaluated Neural Module	Target Class	Precision Index	Recall Rate	Computed F1-Score
PyTorch Multi-Label Network	Active Fire / Flame	0.942	0.915	0.928
PyTorch Multi-Label Network	Smoke Plumes	0.895	0.872	0.883
TensorFlow Sequential Network	Human Recognition	0.968	0.954	0.961
Integrated Ensemble Mode	Co-Occurrence Alert	0.934	0.921	0.927

The experimental results show that the TensorFlow human detection module achieved a strong validation accuracy of 96.85%, reflecting the benefits of the data pre-fetching and dropout regularizations. In the multi-label hazard pipeline, the model achieved an F1-score of 0.928 for active fire classification, while the smoke tracking branch achieved an F1-score of 0.883. This difference highlights the ongoing challenge of segmenting highly translucent, amorphous smoke shapes compared to more distinct flame patterns. Crucially, the specialized data augmentations significantly reduced false positives under complex indoor lighting conditions, allowing the integrated system to reliably distinguish ambient lamps from actual fires while maintaining realtime processing speeds.

7. CONCLUSIONS AND FUTURE WORK

7.1 Project Synthesis

This graduation project successfully demonstrates the implementation of an integrated, real-time computer vision system that simultaneously detects human presence and fire hazards in digital video streams. By using a decoupled ensemble design, the architecture splits the recognition tasks into two specialized pipelines, achieving strong generalization performance without the training trade-offs common in single-backbone networks. The custom PyTorch and TensorFlow networks provide low-latency processing suitable for edge computing deployment, while the multi-class co-occurrence logic introduces an automated risk-assessment layer that identifies when occupants are directly endangered by active hazards.

7.2 Recommendations for Future Research

While the ensemble architecture delivers robust classification performance, several key areas offer opportunities for future enhancement. To further improve deployment efficiency on low-power edge surveillance hardware, future work could explore structural model optimization techniques, such as Post-Training Quantization (PTQ) or network pruning, to reduce parameter density while maintaining accuracy thresholds.

Additionally, the architecture's spatial processing capabilities could be expanded by upgrading the frame extraction backbone to support temporal modeling. Integrating recurrent structures, such as Long Short-Term Memory (LSTM) cells or spatial attention mechanisms, would allow the system to evaluate motion dynamics across consecutive video frames. This temporal context would help distinguish the steady illumination of artificial lights from the flickering motion of open flames, further reducing false alarms and improving tracking stability in highly dynamic environments.

REFERENCES

1. [1] C. Shorten and T. M. Khoshgoftaar, "A survey on Image Data Augmentation for Deep Learning," *Journal of Big Data*, vol. 6, no. 1, pp. 1-48, Dec. 2019.
2. [2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770-778.

3. [3] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019, pp. 8024-8035.

4. [4] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems," arXiv preprint arXiv:1603.04467, 2016.

5. [5] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, vol. 25, pp. 120-125, 2000.

6. [6] T. Boddington, "Early vision-based fire detection algorithms and structural limits under variable domestic illumination scenarios," *Fire Safety Science Journal*, vol. 42, pp. 115-129, 2022.